C: Logistic probabilities and alarm decisions

Problem statement and worked resolution

Companion to `C_budget_violation.ipynb`

How to use this explanation. This resolution separates learning probabilities from choosing an alarm policy. It explains the derivative checks, regularization, cost threshold, and final probability diagnostics. Section numbers follow the notebook's numbered sections where present. Numerical values are rounded and refer to the recorded run, not universal performance claims.

Problem and essential definitions

For an arriving request, estimate the probability of exceeding a latency budget. The binary label is $y = 1$ for a violation and $y = 0$ otherwise. Payload s, queue length q, and cache-hit indicator c are known at arrival. The simulator uses

$$s \sim U(0.5, 10), \quad q \sim \text{Poisson}(3), \quad c \sim \text{Bernoulli}(0.4),$$

$$z = -4 + 0.35s + 0.3q - 1.2c, \qquad \pi = \sigma(z) = \frac{1}{1 + e^{-z}}, \qquad y \mid s, q, c \sim \text{Bernoulli}(\pi).$$

Features are independent here. The score z is real-valued; the sigmoid maps it to a probability. A Bernoulli draw produces 1 with that probability. These labels are generated directly, not by thresholding notebook A's numerical latency. There are 2,000 requests, with simulation seed 17.

Use p for an estimated violation probability, β for all fitted coefficients including the intercept, λ for slope regularization, and t for a decision threshold. Cross-validation (CV) chooses λ; validation chooses t. Gradient descent (GD) optimizes coefficients. These are three distinct operations.

1 Generate data and reserve stratified partitions

`expit` evaluates the sigmoid; `rng.binomial(1, probability)` generates labels. Keep the true probability for simulation diagnostics only, never as an input feature. In practice it is unknown, and final observed latency would not be available at arrival either.

`train_test_split(..., stratify=labels)` preserves approximate class proportions. With split seed 23, obtain 1,200 training, 400 validation, and 400 test rows. Training contains 249 violations, giving prevalence $249/1200 = 0.2075$. The constant baseline predicts this value for every request.

A request with probability 0.20 can still violate. The point of a probability model is to represent uncertainty, not recover every random label. The training mean is a valid baseline because it is available before validation or test outcomes are consulted.

2 Implement stable mean log loss and its gradient

Coordinates and loss

`StandardScaler` learns training means μ_j and scales a_j; use $\tilde{x}_{ij} = (x_{ij} - \mu_j)/a_j$. The design matrix X has a ones column and three standardized feature columns. Thus $z_i = X_i\beta$ and $p_i = \sigma(z_i)$, with $\beta \in \mathbb{R}^4$. For one binary outcome,

$$\ell(y, p) = -y \log p - (1 - y) \log(1 - p).$$

For $y = 1$, assigning probability 0.9 gives loss about 0.105, while 0.1 gives about 2.303. Log loss heavily penalizes confident errors. Natural logarithms are used. The optional regularized objective is

$$J_\lambda(\beta) = \frac{1}{n}\sum_i \ell(y_i, p_i) + \frac{\lambda}{2}\sum_{j=1}^{3}\beta_j^2.$$

The intercept β_0 is excluded. The loss can equivalently be evaluated using the score:

$$\ell(y_i, p_i) = \log\bigl(1 + e^{(1-2y_i)z_i}\bigr).$$

For $y_i = 1$, this is $\log(1 + e^{-z_i})$; for $y_i = 0$, it is $\log(1 + e^{z_i})$. Direct exponentiation can overflow, so `np.logaddexp(0, ...)` computes it stably. The extreme-score check at ± 1000 returns approximately zero for confident correct predictions and 1000 for confident wrong ones, all finite.

Gradient and independent check

Since $\partial\ell/\partial z_i = p_i - y_i$, the gradient is

$$\nabla J_\lambda(\beta) = \frac{1}{n}X^T(p - y) + \lambda(0, \beta_1, \beta_2, \beta_3)^T.$$

`logistic_loss_gradient` implements both objective and gradient. Setting the first component of `penalized_beta` to zero removes intercept penalization from both. For every coordinate j, check a central difference at a nonzero coefficient vector:

$$\frac{\partial J}{\partial \beta_j} \approx \frac{J(\beta + he_j) - J(\beta - he_j)}{2h}, \qquad h = 10^{-5}.$$

Here e_j changes only coordinate j. Checks for $\lambda = 0$ and 0.1 have relative discrepancies about 2×10^{-11}. This validates the derivative calculation; it is not a generalization test.

3 Fit by gradient descent and compare a reference solver

Start at $\beta = 0$, so all initial probabilities are 0.5. Repeat

$$\beta \leftarrow \beta - \eta\nabla J_0(\beta).$$

For unpenalized loss the Hessian is $X^T DX/n$, with $D_{ii} = p_i(1-p_i) \le 1/4$. Therefore $L = \|X\|_2^2/(4n)$ bounds curvature and $\eta = 1/L$ is a safe fixed step. The code stops at gradient norm below 10^{-8}, with a 20,000-update limit.

Main Python calls

`LogisticRegression(C=np.inf, solver="lbfgs", ...)` is the unpenalized reference in the course environment. It receives standardized features without a ones column. `fit` learns coefficients; `predict_proba(... )[:, 1]` selects class-1 probabilities; `intercept_` and `coef_` expose learned parameters. The manual and library fits reach the same solution: 51 GD updates, final gradient norm about 9.6×10^{-9}, and maximum training probability difference about 2.6×10^{-8}. The library solver takes different optimization steps, so its iteration count is not directly comparable to GD's cost per update.

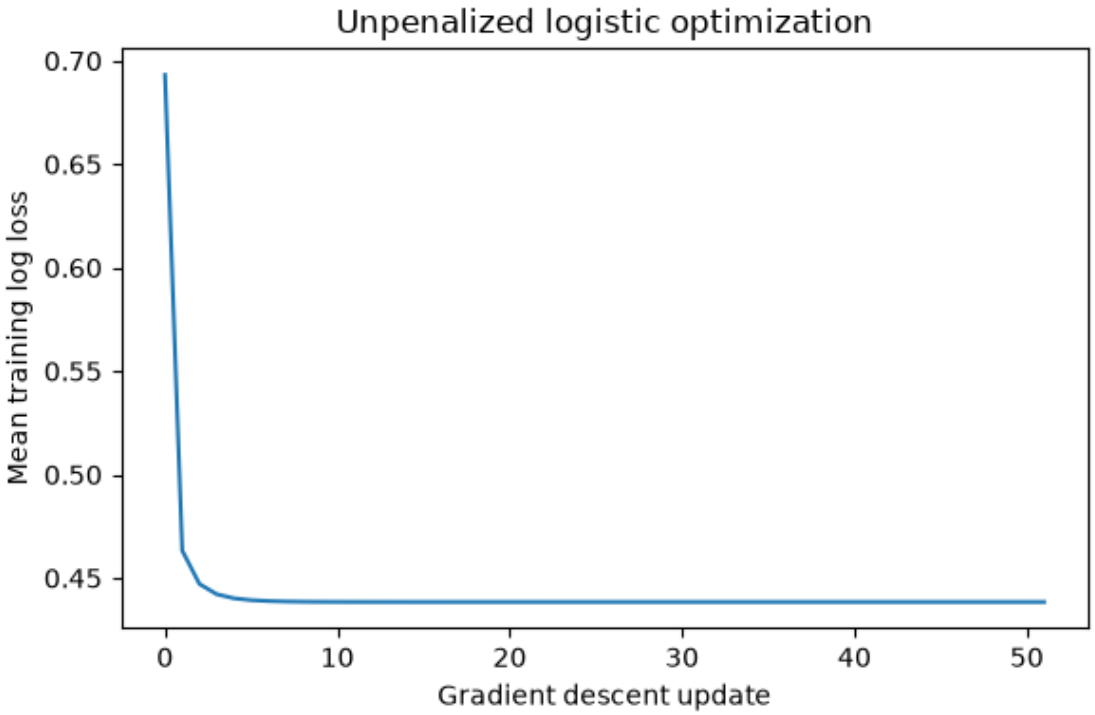


Figure 1: Training log loss decreases as gradient descent learns coefficients. Optimization convergence is distinct from held-out prediction quality.

4 Interpret slopes in original units

Expand the standardized score to obtain

$$w_j = \beta_j / a_j, \qquad b = \beta_0 - \sum_j w_j \mu_j, \qquad z = b + \sum_j w_j x_j.$$

Term	Generating value	Fitted value	Odds multiplier
Intercept	−4	−3.5059	—
Payload	0.35	0.2979	1.347
Queue	0.30	0.2409	1.272
Cache hit	−1.20	−0.9203	0.398

Odds are $p/(1-p)$, and logistic regression models their logarithm. Increasing one original feature by one unit while holding the others fixed multiplies odds by e^{w_j}, not probability by that factor. Cache compares a hit with no hit. For instance, probability 0.20 means odds 0.25. Multiplying by 0.398 gives odds 0.0995 and probability $0.0995/(1+0.0995) \approx 0.0905$. Estimated coefficients differ from generating values because the finite labels are random. These coefficients describe the unpenalized fit; the next step may choose a penalized fit.

5 Select the penalty with training cross-validation

Search $\lambda \in \{0, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1}, 1\}$. `StratifiedKFold` supplies five pairs of row positions, fitting on 960 and scoring on 240 training rows each time. It does not itself train a model. Fit a new scaler inside each fold, apply it to the held-out rows, then fit and score logistic regression. Reuse identical folds across penalties.

`make_logistic` converts our mean-loss penalty to the library convention. The equivalent summed-loss objective is

$$\sum_i \ell(y_i, p_i) + \frac{1}{2C} \sum_{j=1}^{3} \beta_j^2,$$

so $C = 1/(n_{\text{fit}}\lambda)$, with infinity for zero penalty. This C is inverse regularization, not CV or the notebook letter. Recompute the conversion for the 960-row fold fits and the 1,200-row final fit. Score only held-out log loss, without adding the coefficient penalty to the evaluation metric. The selected $\lambda = 0.001$ gives CV loss about 0.440503, versus 0.440529 for zero penalty: a tiny difference, not strong evidence of a decisive advantage. Refit at the selected setting on all training rows and keep this model fixed. Its gradient under the stated penalized objective is near zero, checking the penalty convention numerically.

6 Audit probabilities and choose an alarm threshold

Probability audit

In addition to log loss, calculate the Brier score

$$\text{Brier} = \frac{1}{m} \sum_i (p_i - y_i)^2.$$

Both score probabilities before thresholding; lower is better. Validation gives selected-model log loss about 0.4441 and Brier about 0.1421, compared with baseline values 0.5106 and 0.1644. This audit does not replace the chosen model.

Cost-sensitive decision

A false positive (FP) is an alarm without a violation; a false negative (FN) is a missed violation. The example assigns cost 1 to FP and cost 5 to FN, with zero cost for correct decisions. These are stipulated relative costs, not learned coefficients or a universal standard. If p were the true conditional probability, the expected costs would be

$$\text{alarm} : 1 \times (1-p) + 0 \times p = 1 - p, \qquad \text{no alarm} : 0 \times (1-p) + 5 \times p = 5p.$$

Thus alarm when $1 - p \leq 5p$, or $p \geq 1/6$. The multiplication by 1 represents the false-alarm cost. With general positive costs, the cutoff is $C_{\text{FP}}/(C_{\text{FP}} + C_{\text{FN}})$, assuming correct decisions cost zero. Applying this to fitted probabilities relies on them representing risks appropriately.

The notebook also measures validation cost for a grid of thresholds 0.01 through 0.99, explicitly including 1/6 and 0.5. At each threshold it counts FP and FN and computes $(FP + 5FN)/400$. The graph is therefore *empirical*: the costs are imposed, but its minimizing threshold is observed, not forced. A tie selects the largest threshold. Validation selects exactly 1/6 here, with mean cost 0.525. Another sample or model can select a different cutoff. Freeze this policy without refitting the probability model.

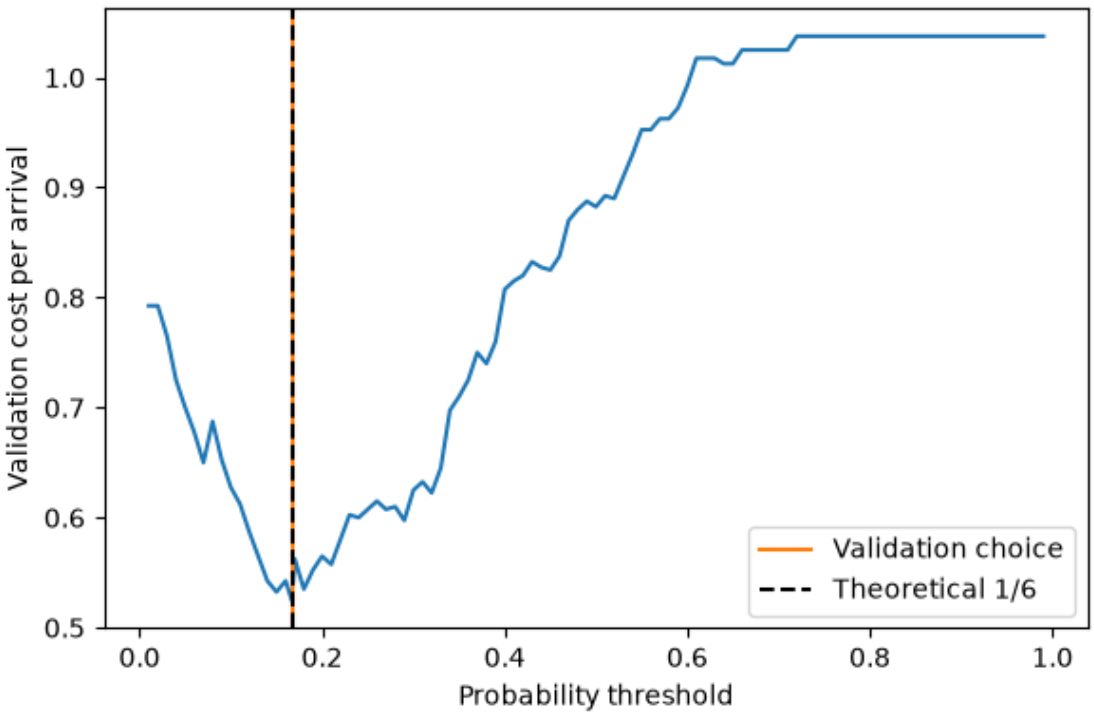


Figure 2: Measured validation cost versus threshold. Including the theoretical cutoff among candidates does not force it to minimize the measured cost.

7 Final test probability and decision evaluation

The test contains 83 violations and 317 nonviolations. The selected model's log loss is about 0.4363 and Brier about 0.1387, versus baseline 0.5106 and 0.1644. The unpenalized model is slightly better on this test; the protocol still keeps the previously selected model.

Threshold	TN	FP	FN	TP	Recall	Mean cost
0.5	310	7	69	14	0.1687	0.8800
1/6	183	134	19	64	0.7711	0.5725

True positives (TP) are detected violations; true negatives (TN) are correctly silent decisions. At 1/6, precision is $TP/(TP+FP) = 64/198 \approx 0.323$, and recall is $TP/(TP+FN) = 64/83 \approx 0.771$. The confusion matrix uses actual labels as rows and predicted labels as columns. The lower threshold catches many more violations but generates more false alarms. Mean cost falls because misses cost five times more. Lower precision can coexist with lower cost. The theoretical and validation-selected policies coincide in this run. Undefined precision or recall is displayed as zero by explicit convention, not treated as evidence from absent cases.

8 Calibration and precision–recall diagnostics

Calibration asks whether requests assigned probability near p exhibit violations at frequency near p. `pd.qcut` divides the 400 test predictions into eight approximately equal-count bins. Compare each mean predicted probability with its observed event fraction. Here all bins contain 50 rows. The first bin has mean probability 0.0445 and 2/50 violations; the last has mean 0.493 and 30/50 violations. Sampling variation is substantial at these sizes.

`precision_recall_curve` varies the threshold of the fixed model. Recall cannot decrease as more alarms are included, but precision need not vary monotonically. The prevalence line, 83/400, is the precision when every request triggers an alarm. The plotted endpoint at zero recall and precision one is a convention, not a useful deployed policy.

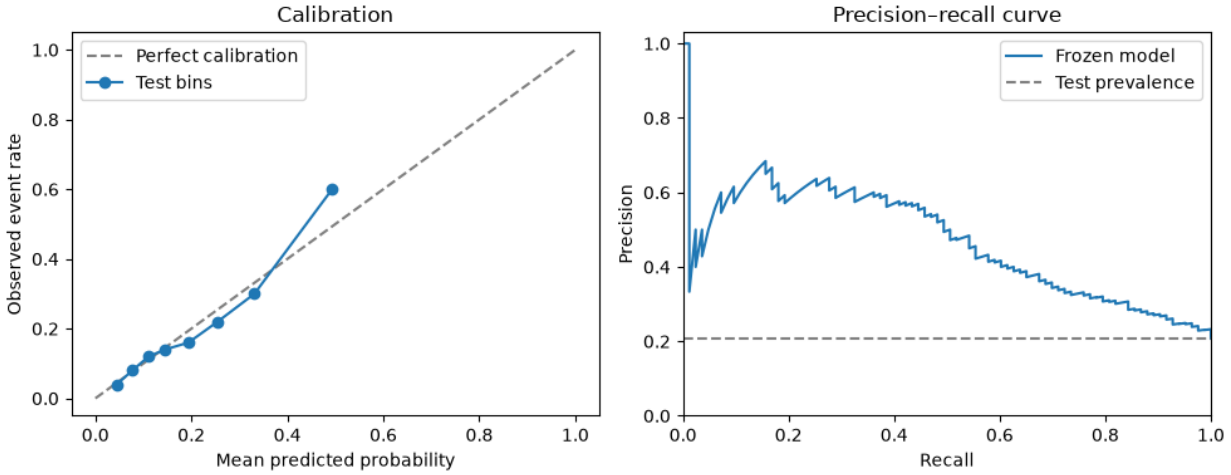


Figure 3: Calibration checks numerical probability levels; the precision–recall curve describes the decision tradeoff across thresholds. The test curve is not used to retune the policy.

Resolution of the interpretation questions

Gradient checking validates derivatives, reference fitting validates optimization, and held-out scores assess prediction: none substitutes for the others. A correct probability model still makes classification errors because labels are stochastic. The costs explain why a lower threshold can be appropriate; they do not make it better for every objective. Calibration deviations may reflect noise, estimation error, or a misspecified model. All conclusions here concern the specified simulation and splits.

Further explanation: probabilities, optimization, and decisions

Why log loss targets a probability rather than a label

Suppose requests with the same inputs have true violation probability π. If a model reports p, its expected log loss for these requests is

$$L(p) = -\pi \log p - (1-\pi)\log(1-p).$$

Differentiating gives $L'(p) = (p-\pi)/(p(1-p))$, so for $0 < \pi < 1$ the unique minimum is at $p = \pi$. For Brier loss, direct expansion gives

$$\mathbb{E}[(Y-p)^2] = \pi(1-\pi) + (p-\pi)^2.$$

The first term is unavoidable random-label variation; the second is probability-estimation error. Both objectives reward honest probabilities, although their penalties for confident mistakes differ. Accuracy alone cannot distinguish predicting 0.51 from 0.99 when both are thresholded to the same label.

A useful derivative cancellation

The score derivative follows from two simple pieces:

$$\frac{\partial \ell}{\partial p} = \frac{p - y}{p(1-p)}, \qquad \frac{dp}{dz} = p(1-p).$$

Their product is $p - y$, explaining the compact gradient in Section 2. Implementing the score-based loss avoids numerically dividing by probabilities close to zero. For a confidently wrong label, the score loss grows approximately linearly with the magnitude of the score; it is large but can still be evaluated without overflow. A derivative check should perturb coefficients, hold data and preprocessing fixed, and avoid excessively tiny finite-difference steps whose subtraction loses precision.

The penalty convention is about sample size as well as strength

At $\lambda = 0.001$, the full-training conversion gives $C = 1/(1200 \times 0.001) = 0.8333\ldots$. Inside a 960-row fold, it gives $C = 1.0417\ldots$. These different C values implement the *same* mean-loss penalty. If instead a study fixes C across folds, that defines a different but legitimate convention; the mistake would be claiming the two conventions are identical. Comparing implementations requires matching loss normalization, feature scaling, and which parameters are penalized, not only matching a parameter name.

How probability errors affect the alarm policy

Let true risk be π and the model's estimate be p. Under the stated costs, the difference between expected alarm and no-alarm costs is $1 - 6\pi$. Choosing the wrong side of the ideal cutoff therefore incurs extra expected cost $|1 - 6\pi|$ for that request. An estimation error that leaves the decision on the correct side causes no change in this two-action cost; an error crossing the cutoff can matter greatly. Thus good average probability loss and good performance under a particular operational cost are related but not identical goals.

The validation curve is a step function in substance: predictions change only when a threshold crosses a predicted score. The notebook draws connecting lines between evaluated grid points for readability; those lines do not mean the cost changes continuously between observations. Selecting the minimum on 400 validation rows is itself a fitted choice. Its apparent advantage over another threshold can disappear on the test set.

Calibration checks need sample sizes

At true risk 0.2, the observed event fraction in 50 independent requests has standard deviation $\sqrt{0.2(1-0.2)/50} \approx 0.057$. A calibration-bin event rate of 0.26 is therefore not automatically strong evidence of a model defect. This calculation is a rough fixed-bin sampling explanation, not a confidence band for the adaptively formed bins in the notebook. Calibration also differs from discrimination: a constant prevalence prediction can be calibrated overall while failing to separate high- and low-risk requests.

Documentation and implementation tips

The LogisticRegression reference is useful for checking the solver, regularization settings, class order, and convergence attributes. Do not assume that column 1 means the desired event for arbitrary labels: inspect `classes_`. `predict_proba` returns probabilities, whereas `predict` returns decisions and `score` reports accuracy. The course pins its library version because options and deprecations evolve.

Practical tips. Keep costs separate from the model's C parameter. Never pass a cost-derived probability cutoff to an uncalibrated score. Examine convergence warnings before interpreting coefficients. Do not treat `max_iter` as a statistical complexity parameter. Finally, preserve the scaler with the fitted model: a coefficient vector by itself is not enough to reproduce standardized-input predictions.